第12章 消息认证码

0. 学习目标总览

教材给出本章十个学习目标：

1. 列举并解释针对消息认证的可能攻击。
2. 定义消息认证码。
3. 列举并解释消息认证码的要求。
4. 简介 HMAC。
5. 简介 CMAC。
6. 解释认证加密的概念。
7. 简介 CCM。
8. 简介 GCM。
9. 讨论密钥封装的概念及其用途。
10. 理解使用哈希函数或消息认证码生成伪随机数的方式。

本章从消息认证面临的攻击出发，先说明认证函数和 MAC 的基本性质，再介绍 HMAC、CMAC、CCM、GCM、密钥封装以及基于哈希函数或 HMAC 的伪随机数生成器。

```
消息认证码
  |
  ├── 认证需求
  |     ├── 防伪装、内容修改、序列修改和计时修改
  |     `-- 与保密性、数字签名的保护边界不同
  |
  ├── 基本认证函数
  |     ├── 哈希函数
  |     ├── 消息加密
  |     `-- MAC：T = MAC(K, M)
  |
  ├── 典型 MAC
  |     ├── HMAC：基于密码学哈希函数
  |     `-- CMAC：基于分组密码
  |
  ├── 认证加密
  |     ├── CCM：CTR + CMAC
  |     `-- GCM：CTR + GHASH
  |
  `-- 其他应用
        ├── 密钥封装
        `-- 基于哈希函数或 HMAC 的 PRNG
```

1. 学习目标 1：针对消息认证的可能攻击

1.1 八类攻击

网络通信环境中可能出现的攻击如下。

攻击	含义
泄露	消息被没有合适密钥的人或程序获得。
通信量分析	敌手分析通信双方的连接频率、持续时间、消息数量和消息长度等通信模式。
伪装	欺诈源插入消息并声称消息来自合法实体，或发送关于收到、未收到消息的欺诈应答。
内容修改	对消息内容进行插入、删除、换位或修改。
序列修改	对消息序列进行插入、删除或重新排序。
计时修改	对消息或会话进行延时和重放。
发送方否认	发送方否认曾经发送过某条消息。
接收方否认	接收方否认曾经收到某条消息。

1.2 不同安全机制的保护边界

攻击范围	所需安全机制
泄露、通信量分析	消息保密性
伪装、内容修改、序列修改、计时修改	消息认证
发送方否认	数字签名
接收方否认	数字签名与专门设计的协议

因此，**消息认证**要验证收到的消息确实来自真正的发送方，并且未被修改；它还可以验证消息的顺序和及时性。数字签名除具有部分或全部消息认证能力外，还可用于抵抗发送方否认。

2. 学习目标 2：消息认证码的定义

2.1 认证函数的三个类型

消息认证或数字签名机制在功能上可分为两层：下层函数生成认证符，上层协议使用该认证符验证消息真实性。生成认证符的函数主要有三类：

1. **哈希函数**：把任意长度的消息映射为定长哈希值。
2. **消息加密**：以整条消息的密文作为认证符。
3. **消息认证码**：以消息和共享密钥为输入，生成定长认证符。

2.2 消息加密与认证

对称加密在一定条件下可以同时提供保密性和认证：只有共享密钥的通信双方能够生成和恢复密文。但接收方仍须能够区分“合法明文”和“随机位串”。如果任意位模式都可作为合法消息，仅凭成功解密并不能证明消息真实。

一种解决方法是在加密前为消息加入可识别结构，例如帧校验序列（FCS）。其顺序非常重要：

- **内部错误控制**：先计算并附加 FCS，再加密 $M \parallel \mathrm{FCS}(M)$。敌手难以生成解密后仍具有正确 FCS 的密文，因此认证能力较强。
- **外部错误控制**：先加密，再为密文计算 FCS。敌手即使不知道明文，也可以为伪造密文重新计算正确的 FCS，因而不能获得同等认证能力。

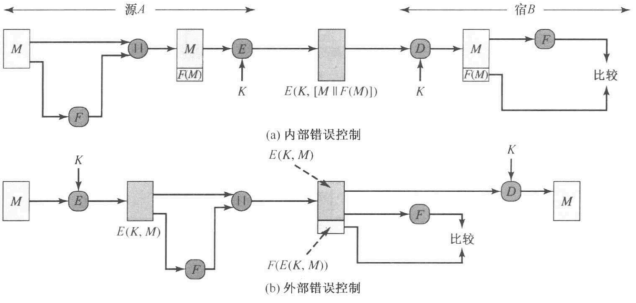


图 12.2　内部和外部错误控制

图 12.2 内部和外部错误控制

直接使用接收方公钥加密只能提供保密性，因为任何人都能使用该公钥生成密文。若发送方用自己的私钥处理消息，则只有发送方能够生成该结果，可提供认证并形成数字签名，但不提供保密性。若还要求保密性，可再用接收方公钥加密，但公钥运算次数随之增加。

2.3 MAC 的定义与验证

消息认证码又称**密码校验和**。设通信双方共享密钥 K，消息为 M，则 MAC 或标记 T 定义为

$$T = \mathrm{MAC}(K, M)$$

其中，M 可以变长，而 T 是定长值。发送方发送 $M \parallel T$；接收方用同一密钥对收到的消息重新计算 MAC，并与收到的标记比较。

在只有通信双方知道密钥且标记一致时，接收方可以相信：

1. 消息未被修改；敌手不知道密钥，不能为修改后的消息生成相符标记。
2. 消息来自真正的发送方；其他参与者不能生成正确的 MAC。
3. 若消息中包含受 MAC 保护的序号，则消息顺序没有被敌手成功修改。

MAC 与加密的主要区别是：MAC 算法不要求可逆。它通常是多对一函数，任意长消息空间中的许多消息会映射到同一个定长标记。

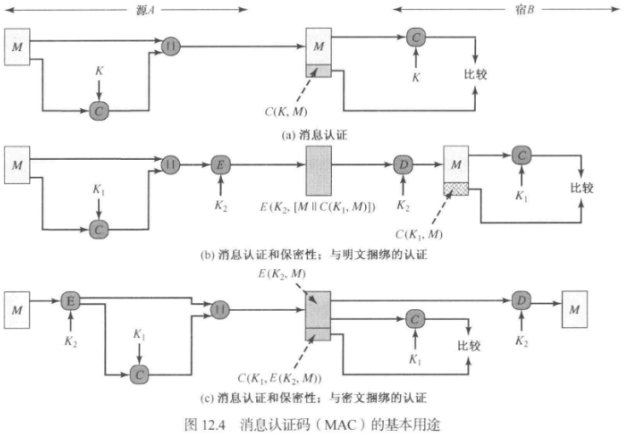


图 12.4　消息认证码（MAC）的基本用途

图 12.4 MAC 的基本用途

仅发送 $M \| T$ 可以提供认证，但不提供保密性。若同时需要保密性，可以对消息和 MAC 一起加密，也可以先加密消息、再对密文计算 MAC。教材指出，一般将 MAC 直接附在明文后再整体加密较好。

MAC 适合与加密分开使用的典型原因包括（前三项必须使用 MAC）：

1. 广播消息只需由特定接收方验证；
2. 高负荷接收方可选择性认证消息；
3. 程序可保持明文形式并按需校验；
4. 某些应用只要求认证而不要求保密；
5. 认证和保密可部署在不同协议层；
6. 消息解密后仍可继续依靠 MAC 保护其完整性。

> 由于发送方和接收方共享同一密钥，双方都能生成相同的 MAC，因此 MAC 不能提供数字签名所要求的不可否认性。

3. 学习目标 3：消息认证码的要求

3.1 三项基本要求

假设敌手知道 MAC 算法但不知道密钥 K，一个安全的 n 位 MAC 应满足：

1. 已知 M 和 $\mathrm{MAC}(K, M)$ 时，构造另一消息 $M' \neq M$，使

   $$\mathrm{MAC}(K, M') = \mathrm{MAC}(K, M)$$

   在计算上不可行。
2. MAC 输出应均匀分布。对随机消息 M 和 M'，标记相等的概率应为 2^{-n}。
3. 若 $M' = f(M)$ 是对 M 的某种已知变换，例如对某些位取反，则仍应有

   $$\Pr[\mathrm{MAC}(K, M) = \mathrm{MAC}(K, M')] = 2^{-n}$$

即消息的任何部分都不应成为认证算法中的“弱点”。

教材用一个反例说明，仅有长密钥并不足以保证安全。若把消息各分组异或后再加密：

$$\Delta(M) = X_1 \oplus X_2 \oplus \cdots \oplus X_m, \qquad \mathrm{MAC}(K, M) = E(K, \Delta(M))$$

敌手可以任意选择前 $m-1$ 个新分组，再调整最后一个分组，使所有分组的异或和保持不变，从而在不知道密钥的情况下沿用原标记完成伪造。这说明 MAC 必须抵抗针对其数学结构的消息变换攻击。

3.2 MAC 的安全性

MAC 的核心安全性质是**抗计算性**：即使敌手已知一个或多个消息-MAC 对，对任何新消息 x，计算有效的 $\mathrm{MAC}(K, x)$ 仍应不可行。

针对 MAC 的攻击分为两类：

- **穷举攻击**：攻击密钥空间需要约 2^k 次操作；直接猜测一个 n 位有效标记需要约 2^n 次操作。
- **密码分析**：利用算法结构寻找比穷举更快的攻击。理想 MAC 应使密码分析攻击的工作量不低于穷举攻击。

因此，穷举攻击的总体强度为

$$\min(2^k, 2^n)$$

其中 k 为密钥长度，n 为 MAC 长度。若要求约 N 位安全强度，应满足

$$\min(k, n) \geq N$$

当 $k > n$ 时，一个已知消息-MAC 对通常会对应多个候选密钥，敌手还需使用更多消息-MAC 对继续筛选。不过，敌手不一定必须恢复密钥；直接伪造标记的复杂度仍受 2^n 限制。

4. 学习目标 4：HMAC

4.1 设计思想与目标

普通密码学哈希函数不使用密钥，不能直接充当 MAC。HMAC 将现有哈希函数作为模块，引入共享密钥构造 MAC。教材列出的设计目标是：

1. 不修改现有哈希函数即可使用。
2. 能方便地替换更快或更安全的哈希函数。
3. 尽量保持原哈希函数的性能。
4. 密钥的使用和处理简单。
5. 可以由嵌入哈希函数的强度分析认证机制的密码分析强度。

把哈希函数视为“黑盒”使 HMAC 易于实现和迁移。当嵌入哈希函数需要替换时，只需更换相应模块，无须重新设计整个 MAC 结构。

4.2 HMAC 结构

设：

- H 为嵌入的哈希函数，分组长度为 b 位，输出长度为 n 位；
- K 为密钥，建议密钥长度至少为 n 位；若 K 长于 b 位，先对其进行哈希；
- K^+ 为把处理后的 K 用 0 填充到 b 位的结果；
- ipad 为十六进制字节 `36` 重复 $b/8$ 次；
- opad 为十六进制字节 `5C` 重复 $b/8$ 次。

HMAC 定义为

$$\mathrm{HMAC}(K, M) = H\left[(K^+ \oplus \mathrm{opad}) \parallel H\left((K^+ \oplus \mathrm{ipad}) \parallel M\right)\right]$$

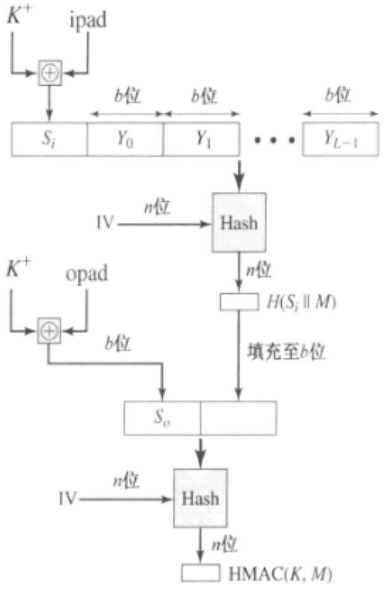


图 12.5　HMAC 的结构

图 12.5 HMAC 的结构

其运算主线是：

```
K 规范化并填充为 K+
  |
  ├── 内层：H((K+ XOR ipad) || M)
  |
  `-- 外层：H((K+ XOR opad) || 内层哈希值)
```

$K^+ \oplus \mathrm{ipad}$ 与 $K^+ \oplus \mathrm{opad}$ 相当于从同一密钥生成两个不同的伪随机前缀。对于长消息，HMAC 与原哈希函数的执行时间大致相同。若预先计算这两个前缀经过压缩函数后的链接值，则只需在密钥改变时重新计算，可进一步降低短消息的处理开销。

4.3 安全性

HMAC 的安全性与嵌入哈希函数的强度存在可分析的关系。教材指出，对 HMAC 的成功攻击等效于下列困难任务之一：

1. 在初始值随机、秘密且未知时计算压缩函数的输出。
2. 在初始值随机且秘密时找到哈希函数碰撞。

一般哈希碰撞可以离线搜索，因为算法和默认初始值公开；攻击 HMAC 时，敌手不知道密钥形成的内部状态，不能以同样方式预先生成所需消息-标记对。这正是 HMAC 相比“直接对消息求哈希”更适合作为认证机制的关键原因。

5. 学习目标 5：CMAC

5.1 DAA 与定长 CBC-MAC 的局限

数据认证算法（DAA）使用 DES 的 CBC 模式，初始向量为 0。将消息分成 64 位分组 $D_1, \ldots, D_N$，不足一组时在右侧填 0：

$$O_1 = E(K, D_1), \qquad O_i = E(K, D_i \oplus O_{i-1})$$

最终数据认证码可取完整的 O_N，也可取其最左侧若干位。该构造对固定长度消息可以是安全的，但不能直接安全地扩展到任意长度。例如，已知单分组消息 X 的 CBC-MAC 为 T，即可构造两分组消息 $X \,\|\, (X \oplus T)$，其 CBC-MAC 仍为 T。

5.2 CMAC 的运算

CMAC 用两个由主加密密钥导出的子密钥区分“完整末分组”和“填充末分组”，从而克服定长 CBC-MAC 的限制。它可使用 AES 或 3DES。

设分组长度为 b，加密密钥为 K。首先计算

$$L = E(K, 0^b), \qquad K_1 = L \cdot x, \qquad K_2 = L \cdot x^2 = (L \cdot x) \cdot x$$

乘法在 $\mathrm{GF}(2^b)$ 上进行。实现时，生成子密钥相当于左移一位，并在满足条件时与由分组长度决定的常数异或。

对消息分组 $M_1, \ldots, M_n$：

- 若最后一组长度恰为 b 位，则令 $M_n' = M_n \oplus K_1$。
- 若最后一组不足 b 位，则在其右侧填充 `1` 和若干 `0`，再令 $M_n' = \mathrm{pad}(M_n) \oplus K_2$。

随后令 $C_0 = 0^b$ 并进行 CBC 链接：

$$C_i = E(K, M_i \oplus C_{i-1}) \quad (1 \le i < n)$$

$$C_n = E(K, M_n' \oplus C_{n-1}), \qquad T = \mathrm{MSB}_{T_{\mathrm{len}}}(C_n)$$

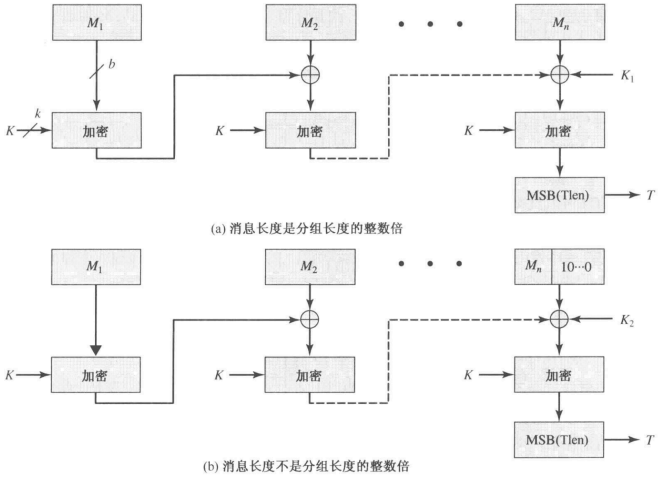


图 12.8　基于密码的消息认证码（CMAC）

图 12.8 CMAC：完整末分组使用 K_1，填充末分组使用 K_2

CMAC 的关键不是简单地对末分组填充，而是用不同子密钥明确区分两种消息结尾，避免不同长度消息之间产生可利用的结构关系。

6. 学习目标 6：认证加密的概念

认证加密（AE） 是同时提供通信保密性和认证（完整性）的加密系统。对消息 M，教材归纳了四种通用组合方式：

方法	发送方处理	接收方顺序	教材示例
先哈希后加密（$H \to E$）	计算 $h = H(M)$，再加密 $M \parallel h$	先解密，后验证	WEP 中采用过类似方法
先认证后加密（$A \to E$）	计算 $T = \mathrm{MAC}(K_1, M)$，再加密 $M \parallel T$	先解密，后验证	SSL/TLS
先加密后认证（$E \to A$）	先得 $C = E(K_2, M)$，再计算 $T = \mathrm{MAC}(K_1, C)$	先验证，后解密	IPsec
独立加密和认证（$E + A$）	分别计算 $C = E(K_2, M)$ 与 $T = \mathrm{MAC}(K_1, M)$	先解密，后验证	SSH

其中后三种方式通常使用两个密钥，分别服务于加密和认证。组合方式本身并不自动保证安全；参数、格式和验证顺序都必须正确设计。CCM 和 GCM 是 NIST 标准化的两种认证加密模式。

7. 学习目标 7：CCM

7.1 组成与输入

CCM 是对独立加密和认证方法的改进，由三部分组成：

- AES 分组密码；
- CTR 模式，用于加密；
- CMAC，用于认证。

加密与 MAC 共用密钥 K。CCM 的输入包括：

1. **明文** P：既要认证又要加密的数据。
2. **关联数据** A：只认证、不加密的数据，例如必须以明文传输的协议首部。
3. **时变值** N：在协议关联的寿命期内，每个实例取不同值，用于阻止重放及相关攻击。

7.2 认证与加密过程

认证阶段将 (N, A, P) 格式化为分组 $B_0, B_1, \ldots, B_r$，再执行类似 CBC-MAC 的链接：

$$Y_0 = E(K, B_0), \qquad Y_i = E(K, B_i \oplus Y_{i-1}) quad (1 \le i \le r)$$

$$T = \mathrm{MSB}_{T_{\mathrm{len}}}(Y_r)$$

加密阶段生成计数器分组 $\mathrm{Ctr}_0, \mathrm{Ctr}_1, \ldots$。$\mathrm{Ctr}_1$ 以后的输出用于 CTR 加密明文，Ctr_0 的输出用于掩蔽认证标记：

$$C_P = P \oplus \mathrm{MSB}_{P_{\mathrm{len}}}(S_1 \,\|\, S_2 \,\|\, \cdots)$$

$$C_T = T \oplus \mathrm{MSB}_{T_{\mathrm{len}}}(S_0)$$

最终输出为 $C = C_P \,\|\, C_T$。

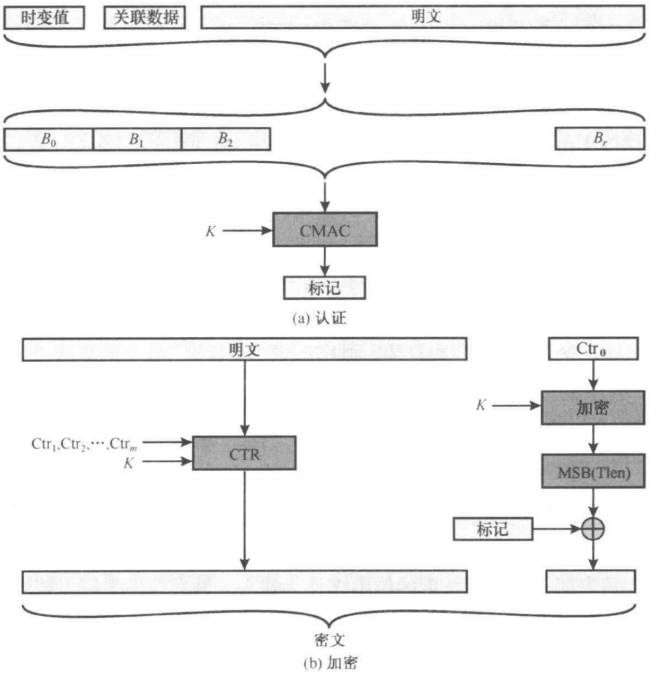


图 12.9　使用分组密码链接–消息认证码（CCM）的计数器

图 12.9 CCM 的认证与加密过程

接收方先用 CTR 恢复明文和标记，再由 (N, A, P) 重新计算 CMAC；若标记不一致则返回无效。CCM 对明文需要两次完整处理：一次计算 MAC，一次执行加密。教材指出其结构较复杂，但已有分析表明它能够提供高强度安全性。

8. 学习目标 8：GCM

8.1 基本特点

Galois/计数器模式（GCM）以 CTR 模式加密消息，并在 $\mathrm{GF}(2^{128})$ 上计算认证值。它采用并行化设计，适合需要高吞吐量、低成本和低延迟的场景。只提供认证而不加密的变体称为 GMAC。

GCM 使用两个核心函数：

- **GHASH**：带哈希密钥的认证函数。
- **GCTR**：计数器每次操作后递增的 CTR 模式。

8.2 GHASH 与 GCTR

设输入 $X = X_1 \parallel X_2 \parallel \cdots \parallel X_m$，每个 X_i 为 128 位，$Y_0 = 0^{128}$。GHASH 的递推为

$$Y_i = (Y_{i-1} \oplus X_i) \cdot H, \qquad 1 \le i \le m$$

返回 Y_m。等价地，

$$\mathrm{GHASH}_H(X) = X_1 H^m \oplus X_2 H^{m-1} \oplus \cdots \oplus X_m H$$

其中乘法在 $\mathrm{GF}(2^{128})$ 上进行。H 的幂可以预计算，各输入分组的乘法也可并行处理。

GCTR 对每个 128 位计数器分组加密，再与输入异或；最后一个不足 128 位的分组只取所需的最高有效位。计数器只递增最右侧 32 位，因此计数器值能快速生成，加密运算可以并行执行。

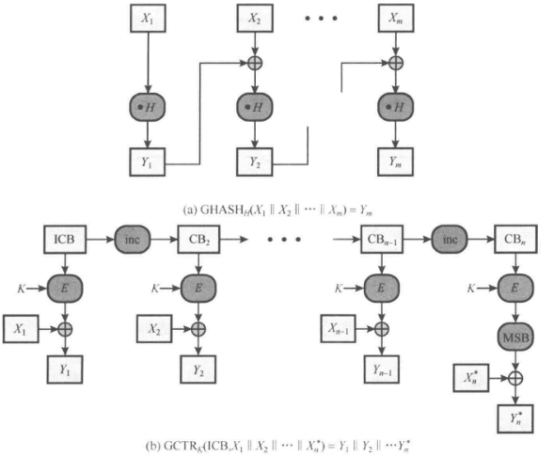


图 12.10　GCM 认证和加密函数

图 12.10 GCM 的 GHASH 与 GCTR 函数

8.3 GCM 认证加密主线

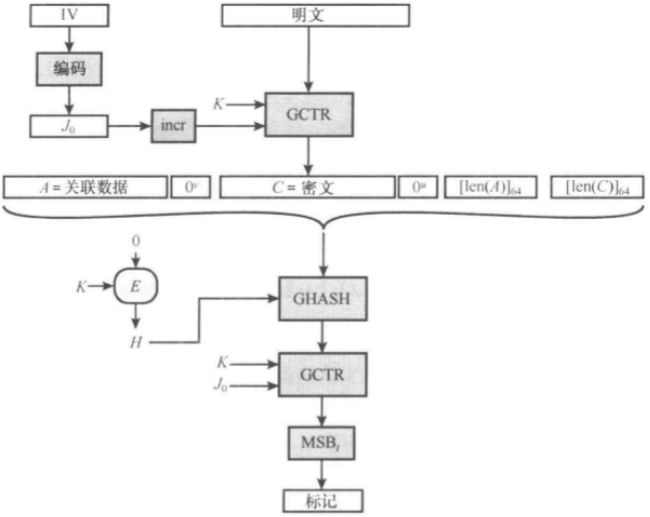


图 12.11　Galois/计数器-消息认证码（GCM）

图 12.11 Galois/计数器-消息认证码(GCM)

输入为密钥 K、初始向量 IV、明文 P 和附加认证数据 A。首先生成哈希密钥

$$H = E(K, 0^{128})$$

再由 IV 形成预计数器分组 J_0。当 IV 长度为 96 位时，

$$J_0 = IV \parallel 0^{31} \parallel 1$$

否则，对填充后的 IV 及其长度字段执行 GHASH 得到 J_0。随后：

$$C = \text{GCTR}_K(\text{inc}_{32}(J_0), P)$$

将关联数据 A、密文 C、必要的 0 填充以及二者的 64 位长度字段连接后进行 GHASH，得到 S：

$$S = \text{GHASH}_H\left(A \parallel 0^v \parallel C \parallel 0^u \parallel [\text{len}(A)]_{64} \parallel [\text{len}(C)]_{64}\right)$$

最后生成 t 位认证标记：

$$T = \text{MSB}_t\left(\text{GCTR}_K(J_0, S)\right)$$

GCM 的效率来自两方面：CTR 加密可以并行，GHASH 只使用易于硬件实现的 Galois 域乘法。关联数据虽不加密，但被纳入标记计算，因此其完整性同样受保护。

9. 学习目标 9：密钥封装

9.1 概念与用途

密钥封装（KW）使用通信双方预先共享的 **密钥加密密钥（KEK）** 安全交换新的对称密钥。NIST 的密钥封装模式可以使用 AES 或三重 DES，AES 版本也见于 RFC 3394。

之所以需要专门的工作模式，是因为待封装密钥可能长于分组密码的分组长度。例如 AES 的分组长度为 128 位，而密钥可为 192 位或 256 位。密钥会反复用于保护大量数据，一旦泄露，影响通常比普通数据分组泄露更严重，因此封装模式要求输出各位以复杂方式依赖全部输入位。

与一般消息或文件加密相比，密钥封装吞吐量较低，但它通常只处理少量密钥数据，这一折中在密钥管理中可以接受。

9.2 封装过程

密钥封装算法按 64 位处理密钥数据。设明文密钥分为 n 个分组 $P_1, \ldots, P_n$，阶段数为 $s = 6n$，初始完整性校验值为

$$A(0) = \texttt{A6A6A6A6A6A6A6A6}$$

并令 $R(0, i) = P_i$。每一阶段 t 的主线为：

$$W = E\left(K, A(t-1) \parallel R(t-1, 1)\right)$$

$$A(t) = \mathrm{MSB}_{64}(W) \oplus [t]_{64}, \qquad R(t, n) = \mathrm{LSB}_{64}(W)$$

其余 R 寄存器向前移动。完成 s 个阶段后，输出

$$(C_0, C_1, \ldots, C_n) = (A(s), R(s, 1), \ldots, R(s, n))$$

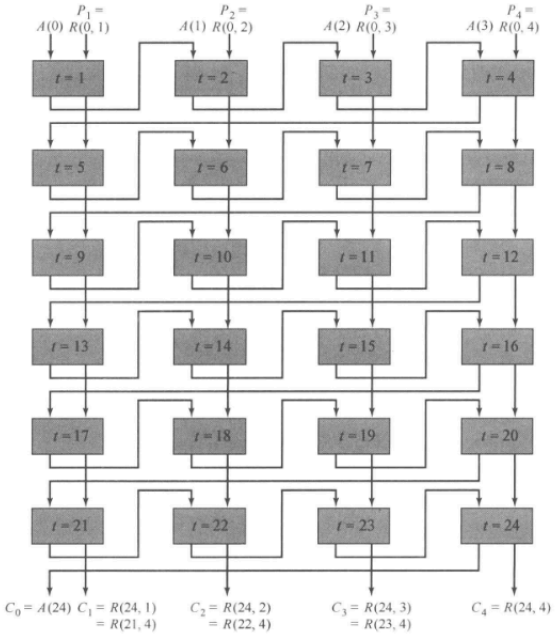


图 12.12　对 256 位密钥的密钥封装操作

图 12.12 对 256 位密钥的密钥封装操作

为了保存 64 位完整性校验值，密文比明文密钥多一个 64 位分组。多阶段反馈和寄存器移动增强了雪崩效应及输入位之间的混淆。

> **理解**
>
> 4 个槽，初始化将 256 位明文分成 64 位的 4 个分组分别存储进 4 个 64 位寄存器，另外初始化完整性校验值（ICV）寄存器 R_0 为固定常量 `A6A6A6A6A6A6A6A6` 。随后执行 6 轮迭代，每轮对 n 个寄存器按序执行 AES 加密操作：将 R_0 与 R_i 拼接加密，加密结果的高 64 位与轮常数异或后更新 R_0 ，低 64 位更新 R_i。这样，原始的 ICV 值会在迭代过程中不断被后续数据和轮常数混合，最终封密钥文的前 64 位即包含整个密钥的完整性校验信息，接收方在解密时可通过重新计算 ICV 来验证密钥的完整性与来源。

9.3 解封与完整性验证

密钥解封按阶段逆序使用解密函数，恢复 $A(0)$ 和各明文分组。若恢复的 $A(0)$ 不等于固定 ICV `A6A6A6A6A6A6A6A6` ，则说明密文出现错误或变化，必须拒绝恢复出的明文密钥。

因此，密钥封装不仅提供密钥数据的保密性，还通过 ICV 验证提供数据完整性。封装与解封互逆的关键是：阶段编号 t 的异或值在解封时被抵消，且分组密码的解密运算恢复前一阶段的 A 与 R。

10. 学习目标 10：使用哈希函数和 MAC 的 PRNG

10.1 基本思想

伪随机数生成器（PRNG）由种子和确定性算法组成。若输出用于生成会话密钥等伪随机函数结果，种子必须只有使用者知道；若用于流密码，种子就是通信双方共享的秘密密钥。

分组密码、密码学哈希函数和 MAC 的输出都具有类似随机序列的性质，因此都可以作为 PRNG 的构造基础。教材介绍了基于哈希函数与基于 HMAC 的两类方案。

10.2 基于哈希函数的 PRNG

设种子状态为 V，长度为 $\mathrm{seedlen}$；所需安全强度为 K 位时，要求 $\mathrm{seedlen} \geq K + 64$。哈希函数每次输出 outlen 位，要求生成 N 位随机串，则

$$m = \left\lceil \frac{N}{\mathrm{outlen}} \right\rceil$$

重复计算

$$w_i = H(V), \qquad V = (V + 1) \bmod 2^{\mathrm{seedlen}}$$

最后连接 $w_1 \parallel w_2 \parallel \cdots \parallel w_m$，并在需要时截断最后一个分组。SP 800-90 还规定周期性更新 V 。教材指出，对于 SHA-2 等强密码学哈希算法，目前没有发现这种方案的已知或可疑缺陷。

10.3 基于 HMAC 的 PRNG

基于 MAC 的方案通常使用 HMAC，输入为密钥 K 和种子 V。SP 800-90 的基本链式结构为

$$w_0 = V, \qquad w_i = \mathrm{HMAC}(K, w_{i-1}) quad (1 \leq i \leq m)$$

各输出分组连接后截取所需的 N 位。对每个输出分组，密钥保持不变，下一分组的输入等于前一分组的 HMAC 输出；标准还规定周期性更新 K 和 V。

运行开销也会加倍。

NIST SP 800-90	IEEE 802.11i	TLS/WTLS
$m = \lceil n / \text{outlen} \rceil$ $w_0 = V$ $W =$ 空字符串 for $i = 1$ to m $w_i = \text{MAC}(K, w_{i-1})$ $W = W \| w_i$ 返回 W 最左侧的 n 位	$m = \lceil n / \text{outlen} \rceil$ $W =$ 空字符串 for $i = 1$ to m $w_i = \text{MAC}(K, (V \| i))$ $W = W \| w_i$ 返回 W 最左侧的 n 位	$m = \lceil n / \text{outlen} \rceil$ $A(0) = V$ $W =$ 空字符串 for $i = 1$ to m $A(i) = \text{MAC}(K, A(i-1))$ $w_i = \text{MAC}(K, (A(i) \| V))$ $W = W \| w_i$ 返回 W 最左侧的 n 位

图 12.15　基于 HMAC 的三个 PRNG

图 12.15 基于 HMAC 的三个 PRNG

教材比较了三种基于 HMAC 的构造：

方案	每个输出分组的输入特点
NIST SP 800-90	前一输出分组作为下一次 HMAC 的输入。
IEEE 802.11i	种子与递增计数器连接后作为输入。
TLS/WTLS	先计算中间链 $A(i)$，再对 $A(i) \| V$ 调用 HMAC。

基于 HMAC 的方案每个输出分组通常需要两次哈希函数运算，因此比直接使用哈希函数慢；教材认为其优点是更令人放心，并可提供更高的安全性。即使敌手观察到链式结构中的输入和输出，只要 HMAC 安全，就不能恢复密钥 K，也不能预测后续随机位。

本章关键术语整理

术语	核心含义
消息认证	验证消息来源、完整性，并可验证顺序和及时性。
消息认证码（MAC）	由共享密钥和消息生成的定长认证符。
标记（Tag）	MAC 或认证加密算法输出的认证值。
抗计算性	已知若干消息-MAC 对时，仍无法为新消息计算有效 MAC。
HMAC	使用内外两层带密钥哈希构造的 MAC。
ipad / opad	HMAC 中形成内层、外层密钥前缀的固定填充值。
DAA	基于 DES-CBC 的旧式数据认证算法，只适合固定长度消息。
CMAC	用分组密码和两个派生子密钥处理任意长度消息的 MAC。
认证加密（AE）	同时提供保密性和认证的加密系统。
关联数据	在认证加密中只认证、不加密的数据。
CCM	组合 CTR 加密与 CMAC 认证的认证加密模式。
GCM	组合 CTR 加密与 Galois 域哈希认证的并行认证加密模式。
GHASH	GCM 中在 $\text{GF}(2^{128})$ 上计算的带密钥哈希函数。
GCTR	GCM 中使用的计数器模式函数。

术语	核心含义
密钥封装（KW）	使用 KEK 对新的对称密钥同时实施保密性和完整性保护。
密钥加密密钥（KEK）	用于封装其他密钥的预共享对称密钥。
完整性校验值（ICV）	密钥解封后用于判断封装数据是否被改变的固定值。
PRNG	由秘密种子和确定性算法生成伪随机位流的机制。

学习目标自检

1. 能否列出八类通信攻击，并说明保密性、消息认证和数字签名各自主要抵抗哪些攻击？
2. 能否写出 $T = \mathrm{MAC}(K, M)$，并说明 MAC 可以证明什么、不能证明什么？
3. 能否解释安全 MAC 的三项基本要求，以及其穷举攻击强度为何是 $\min(2^k, 2^n)$？
4. 能否写出 HMAC 的完整公式，并解释 K^+、ipad 和 opad 的作用？
5. 能否说明定长 CBC-MAC 的弱点，以及 CMAC 如何用 K_1、K_2 区分两类末分组？
6. 能否区分 $H \to E$、$A \to E$、$E \to A$ 和 $E + A$ 四种认证加密组合方式？
7. 能否说明 CCM 的三个输入，并概括 CMAC 认证和 CTR 加密的处理流程？
8. 能否说明 GCM 中 GHASH、GCTR、H 和 J_0 的作用，并写出密文与标记的核心公式？
9. 能否解释密钥封装为何需要固定 ICV，以及封装输出为何比输入多一个 64 位分组？
10. 能否比较基于哈希函数和基于 HMAC 的 PRNG 在状态更新、性能与安全性方面的差异？